

~\OneDrive\Desktop\artificial neural networks\Rbm2.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # -----
5 # Data: 3-bit XOR
6 # -----
7 def xor3_patterns():
8     """Return the 4 odd-parity (XOR=1) patterns in  $\pm 1$  encoding."""
9     return np.array([
10         [-1, -1, -1],
11         [-1, +1, +1],
12         [+1, -1, +1],
13         [+1, +1, -1]
14     ], dtype=np.int8)
15
16 def pattern_to_index(v):
17     """
18     Map a  $\pm 1$  vector [v1, v2, v3] to an integer index 0..7 (for arrays/plots).
19     Example: [1, -1, -1] -> 100b -> 4 ; [-1, 1, -1] -> 010b -> 2
20     """
21     idx = 0
22     if v[0] == 1: idx += 4
23     if v[1] == 1: idx += 2
24     if v[2] == 1: idx += 1
25     return idx
26
27 def data_distribution_xor():
28     """p_data over the 8 visible states: 4 XOR-odd states have prob 1/4, others 0."""
29     p = np.zeros(8, dtype=float)
30     for v in xor3_patterns():
31         p[pattern_to_index(v)] = 0.25
32     return p
33
34 def prob_plus_one(field):
35     """
36     For  $\pm 1$  neurons:  $P(+1 \mid b) = (1 + \tanh(b)) / 2 = 1 / (1 + \exp(-2b))$ .
37     Input: field (local field). Output: probability of +1.
38     """
39     return (1.0 + np.tanh(field)) / 2.0
40
41 def all_pm1(n):
42     """
43     Generate all  $2^n$  vectors of length n with entries in  $\{-1, +1\}$ .
44     Uses //2 to peel bits (simple and beginner-friendly).
45     """
46     patterns = []
47     for i in range(2**n):
48         x = i
49         vec = []
50         for _ in range(n):
51             bit = x % 2
```

```
52         vec.append(+1 if bit == 1 else -1)
53         x = x // 2
54         patterns.append(vec)
55     return np.array(patterns, dtype=np.int8)
56
57
58 # -----
59 # RBM class ( $\pm 1$  units)
60 # -----
61 class RBM:
62     """
63     Restricted Boltzmann Machine with  $\pm 1$  neurons.
64     Algorithm 3 (CD-k) with tanh fields for updates.
65     Convention: weights shape = (n_visible, n_hidden):
66         b_h = v @ W - bias_hidden,    b_v = h @ W^T - bias_visible
67     """
68     def __init__(self, n_visible=3, n_hidden=1, seed=None):
69         self.n_visible = n_visible
70         self.n_hidden = n_hidden
71         self.rng = np.random.default_rng(seed)
72         #  $W \sim N(0,1)$ 
73         self.weights = self.rng.normal(0.0, 1.0, size=(n_visible, n_hidden))
74         # biases = 0
75         self.bias_visible = np.zeros(n_visible, dtype=float)
76         self.bias_hidden = np.zeros(n_hidden, dtype=float)
77
78     # ----- Conditional sampling -----
79     def sample_hidden_given_visible(self, v):
80         field_h = v @ self.weights - self.bias_hidden           # (M,)
81         prob_h = prob_plus_one(field_h)                         # (M,)
82         h = np.ones(self.n_hidden, dtype=np.int8)
83         u = self.rng.random(self.n_hidden)
84         for i in range(self.n_hidden):
85             if u[i] > prob_h[i]:
86                 h[i] = -1
87         return h, field_h
88
89     def sample_visible_given_hidden(self, h):
90         field_v = h @ self.weights.T - self.bias_visible       # (N,)
91         prob_v = prob_plus_one(field_v)                         # (N,)
92         v = np.ones(self.n_visible, dtype=np.int8)
93         u = self.rng.random(self.n_visible)
94         for j in range(self.n_visible):
95             if u[j] > prob_v[j]:
96                 v[j] = -1
97         return v, field_v
98
99     # ----- Energy and exact model probabilities -----
100    def energy(self, v, h):
101        """ $H(v,h) = - \sum_{ij} W_{ij} v_i h_j + \sum_i bV_i v_i + \sum_j bH_j h_j$ """
102        return - (v @ self.weights @ h) + np.dot(self.bias_visible, v) +
103        np.dot(self.bias_hidden, h)
104
105    def model_probs_exact(self):
```

```
105 """Compute P_model(v) exactly by summing over all hidden states."""
106 V = all_pm1(self.n_visible)      # (8, 3)
107 H = all_pm1(self.n_hidden)      # (2^M, M)
108 unnorm = np.zeros(len(V), dtype=float)
109 for i, v in enumerate(V):
110     s = 0.0
111     for h in H:
112         s += np.exp(-self.energy(v, h))
113     unnorm[i] = s
114 Z = unnorm.sum()
115 probs = unnorm / Z
116 out = np.zeros(2 ** self.n_visible, dtype=float)
117 for i, v in enumerate(V):
118     out[pattern_to_index(v)] = probs[i]
119 return out
120
121 def model_probs_by_gibbs(self, n_samples=100_000, burn_in=10_000):
122     """Estimate P_model(v) via a long Gibbs chain and frequencies."""
123     v = self.rng.choice([-1, 1], size=self.n_visible).astype(np.int8)
124     for _ in range(burn_in):
125         h, _ = self.sample_hidden_given_visible(v)
126         v, _ = self.sample_visible_given_hidden(h)
127     counts = np.zeros(2 ** self.n_visible, dtype=np.int64)
128     for _ in range(n_samples):
129         h, _ = self.sample_hidden_given_visible(v)
130         v, _ = self.sample_visible_given_hidden(h)
131         counts[pattern_to_index(v)] += 1
132     return counts / counts.sum()
133
134 # ----- One CD-k update on a single data pattern -----
135 def cd_k_update_single(self, x, k, eta):
136     v0 = x.copy()
137     h0, field_h0 = self.sample_hidden_given_visible(v0)
138     v, h, field_h = v0, h0, field_h0
139     for _ in range(k):
140         v, _ = self.sample_visible_given_hidden(h)
141         h, field_h = self.sample_hidden_given_visible(v)
142     tanh_h0 = np.tanh(field_h0)
143     tanh_hk = np.tanh(field_h)
144     # weights
145     for m in range(self.n_hidden):
146         for n in range(self.n_visible):
147             self.weights[n, m] += eta * (tanh_h0[m] * v0[n] - tanh_hk[m] * v[n])
148     # visible biases
149     for n in range(self.n_visible):
150         self.bias_visible[n] += -eta * (v0[n] - v[n])
151     # hidden biases
152     for m in range(self.n_hidden):
153         self.bias_hidden[m] += -eta * (tanh_h0[m] - tanh_hk[m])
154
155 # ----- Training loop (mini-batch p0 = batch_size) -----
156 def train(self, data_patterns, k=10, eta=0.005, epochs=10_000, batch_size=20,
157 verbose_every=0):
158     data = np.array(data_patterns, dtype=np.int8)
```

```
158         for ep in range(epochs):
159             idx = self.rng.integers(0, len(data), size=batch_size)
160             batch = data[idx]
161             for x in batch:
162                 self.cd_k_update_single(x, k=k, eta=eta)
163             if verbose_every and (ep + 1) % verbose_every == 0:
164                 print(f" epoch {ep+1}/{epochs}")
165
166
167 # -----
168 # Metrics: D_KL and theory bound
169 # -----
170 def kl_divergence(p_data, p_model, eps=1e-12):
171     """D_KL(p||q) = sum_i p(i) * log( p(i) / q(i) ), natural log."""
172     s = 0.0
173     for i in range(len(p_data)):
174         if p_data[i] > 0:
175             s += p_data[i] * (np.log(p_data[i]) - np.log(max(p_model[i], eps)))
176     return float(s)
177
178 def theoretical_bound_eq_4_40(M, N=3):
179     """Equation (4.40) upper bound for D_KL as a function of M (hidden units)."""
180     threshold = (2 ** (N - 1)) - 1
181     if M >= threshold:
182         return 0.0
183     m1 = int(np.floor(np.log2(M + 1)))
184     val = N - m1 - (M + 1) / (2 ** m1)
185     return max(0.0, np.log(2.0) * val)
186
187
188 # -----
189 # Fig-4.3-style scatter: one figure per (preset, p0, mode)
190 # -----
191 def plot_dkl_scatter_vs_hidden(preset="IDEAL",
192                                learning_rate=0.005,
193                                epochs=None, # None → from preset
194                                k=None, # None → from preset
195                                batch_sizes=(20,), # tuple/list of p0 values
196                                n_runs=20, # runs per M
197                                use_gibbs_sampling=False, # dots from Gibbs or Exact
198                                n_gibbs_samples=50_000,
199                                gibbs_burn_in=10_000,
200                                base_seed=1234):
201     # Preset defaults
202     if preset.upper() == "FAST":
203         k_def, _, epochs_def = 1, 0.005, 3000
204     else:
205         k_def, _, epochs_def = 10, 0.005, 10_000
206
207     if k is None: k = k_def
208     if epochs is None: epochs = epochs_def
209     eta = learning_rate
210
211     M_values = [1, 2, 4, 8]
```

```
212     p_data = data_distribution_xor()
213
214     for p0 in batch_sizes:
215         dkl_by_M = {M: [] for M in M_values}
216         for M in M_values:
217             for r in range(n_runs):
218                 seed = base_seed + 100*M + r
219                 rbm = RBM(n_visible=3, n_hidden=M, seed=seed)
220                 rbm.train(xor3_patterns(), k=k, eta=eta,
221                         epochs=epochs, batch_size=p0, verbose_every=0)
222
223                 if use_gibbs_sampling:
224                     p_model = rbm.model_probs_by_gibbs(n_samples=n_gibbs_samples,
225                                                         burn_in=gibbs_burn_in)
226                 else:
227                     p_model = rbm.model_probs_exact()
228
229                 dkl_by_M[M].append(kl_divergence(p_data, p_model))
230
231     # ---- plotting ----
232     plt.figure(figsize=(7.8, 3.9))
233     M_cont = np.linspace(0.8, 8.2, 200)
234     upper_bound_vals = [theoretical_bound_eq_4_40(int(m)) for m in M_cont]
235     plt.plot(M_cont, upper_bound_vals, color='C0', lw=2,
236             label='Upper bound (Eq. 4.40)')
237
238     # scatter clouds + highlight min per M
239     for M in M_values:
240         dkl_vals = np.asarray(dkl_by_M[M], dtype=float)
241         x_jitter = M + np.random.normal(0, 0.03, size=dkl_vals.size)
242         plt.scatter(x_jitter, dkl_vals, s=18, alpha=0.65, color='k')
243     mins = [min(dkl_by_M[M]) for M in M_values]
244     plt.scatter(M_values, mins, s=36, color='k', zorder=3)
245
246     mode = "Gibbs" if use_gibbs_sampling else "Exact"
247     plt.xlabel('M (number of hidden units)')
248     plt.ylabel(r'$D_{KL}(p_{\mathrm{data}} \parallel p_{\mathrm{model}})$')
249     plt.title(f'D_KL scatter vs M – preset={preset}, k={k}, η={eta}, '
250             f'epochs={epochs}, p0={p0} [{mode}]')
251     plt.grid(alpha=0.25)
252     plt.legend(loc='upper right')
253     plt.tight_layout()
254     plt.show()
255
256
257 # -----
258 # NEW: overlay of learning rates (lines through best D_KL)
259 # -----
260 def compare_learning_rates(preset="IDEAL",
261                           etas=(0.001, 0.005, 0.01),
262                           epochs=None, k=None, batch_size=20,
263                           n_runs=10, base_seed=2222):
264     # Preset defaults
265     if preset.upper() == "FAST":
```

```
266     k_def, _, epochs_def = 1, 0.005, 3000
267 else:
268     k_def, _, epochs_def = 10, 0.005, 10_000
269 if k is None:     k = k_def
270 if epochs is None: epochs = epochs_def
271
272 M_values = [1, 2, 4, 8]
273 p_data = data_distribution_xor()
274
275 plt.figure(figsize=(8.6, 4.6))
276 M_cont = np.linspace(0.8, 8.2, 200)
277 plt.plot(M_cont, [theoretical_bound_eq_4_40(int(m)) for m in M_cont],
278          label='Upper bound (Eq. 4.40)')
279
280 for eta in etas:
281     best_per_M = []
282     for M in M_values:
283         vals = []
284         for r in range(n_runs):
285             seed = base_seed + 100*M + r
286             rbm = RBM(n_visible=3, n_hidden=M, seed=seed)
287             rbm.train(xor3_patterns(), k=k, eta=eta,
288                      epochs=epochs, batch_size=batch_size, verbose_every=0)
289             p_model = rbm.model_probs_exact()
290             vals.append(kl_divergence(p_data, p_model))
291             best_per_M.append(min(vals))
292         plt.plot(M_values, best_per_M, marker='o', linestyle='--',
293                  label=f'η={eta} (k={k}, epochs={epochs}, p0={batch_size})')
294
295 plt.xlabel('M (number of hidden units)')
296 plt.ylabel(r'$D_{KL}(p_{\mathrm{data}} \parallel p_{\mathrm{model}})$')
297 plt.title(f'Learning rate sweep – preset={preset}')
298 plt.grid(alpha=0.25)
299 plt.legend()
300 plt.tight_layout()
301 plt.show()
302
303
304 # ---- runner ----
305 if __name__ == "__main__":
306     # 1) IDEAL (k=10) – Exact
307     plot_dkl_scatter_vs_hidden(preset="IDEAL",
308                               learning_rate=0.005,
309                               batch_sizes=(20,),
310                               n_runs=20,
311                               use_gibbs_sampling=False)
312
313     # 2) IDEAL (k=10) – Gibbs
314     plot_dkl_scatter_vs_hidden(preset="IDEAL",
315                               learning_rate=0.005,
316                               batch_sizes=(20,),
317                               n_runs=20,
318                               use_gibbs_sampling=True,
319                               n_gibbs_samples=50_000,
```

```
320         gibbs_burn_in=10_000)
321
322     # 3) FAST (k=1) – Exact
323     plot_dkl_scatter_vs_hidden(preset="FAST",
324                               learning_rate=0.005,
325                               batch_sizes=(20,),
326                               n_runs=20,
327                               use_gibbs_sampling=False)
328
329     # 4) Learning-rate sweep (overlay lines, Exact)
330     compare_learning_rates(preset="IDEAL",
331                           etas=(0.001, 0.005, 0.01),
332                           epochs=None,      # uses preset default
333                           k=None,           # uses preset default
334                           batch_size=20,
335                           n_runs=10)
```